

데이터 과학 및 분석

의사결정트리, 랜덤포레스트, XGBoost

호서대학교 조학수

01 의사결정트리

02 앙상블과 배깅

03 랜덤포레스트와 XGBoost

04 스택킹 앙상블

01 의사결정트리

분 류 (Classification)

분류(Classification)알고리즘은 지도학습의 일종으로, 기존에 존재하는 데이터의 Category 관계를 파악하고, 새롭게 관측된 데이터의 Category를 스스로 판별하는 알고리즘이다. 서로 다른 클래스의 데이터가 잘 구분되게 하는 규칙을 찾는 것을 공통적인 학습 목표로 한다.



그림 출처: <https://opentutorials.org/course/4548/28942>

01 의사결정트리

분류 알고리즘의 종류

분류(Classification)은 기계학습 분야에서 가장 활용도가 높은 알고리즘으로, 분류를 위한 다양한 알고리즘이 존재하는데, 여기에서는 의사결정 나무 알고리즘을 통해 분류 알고리즘의 원리를 알아본다.

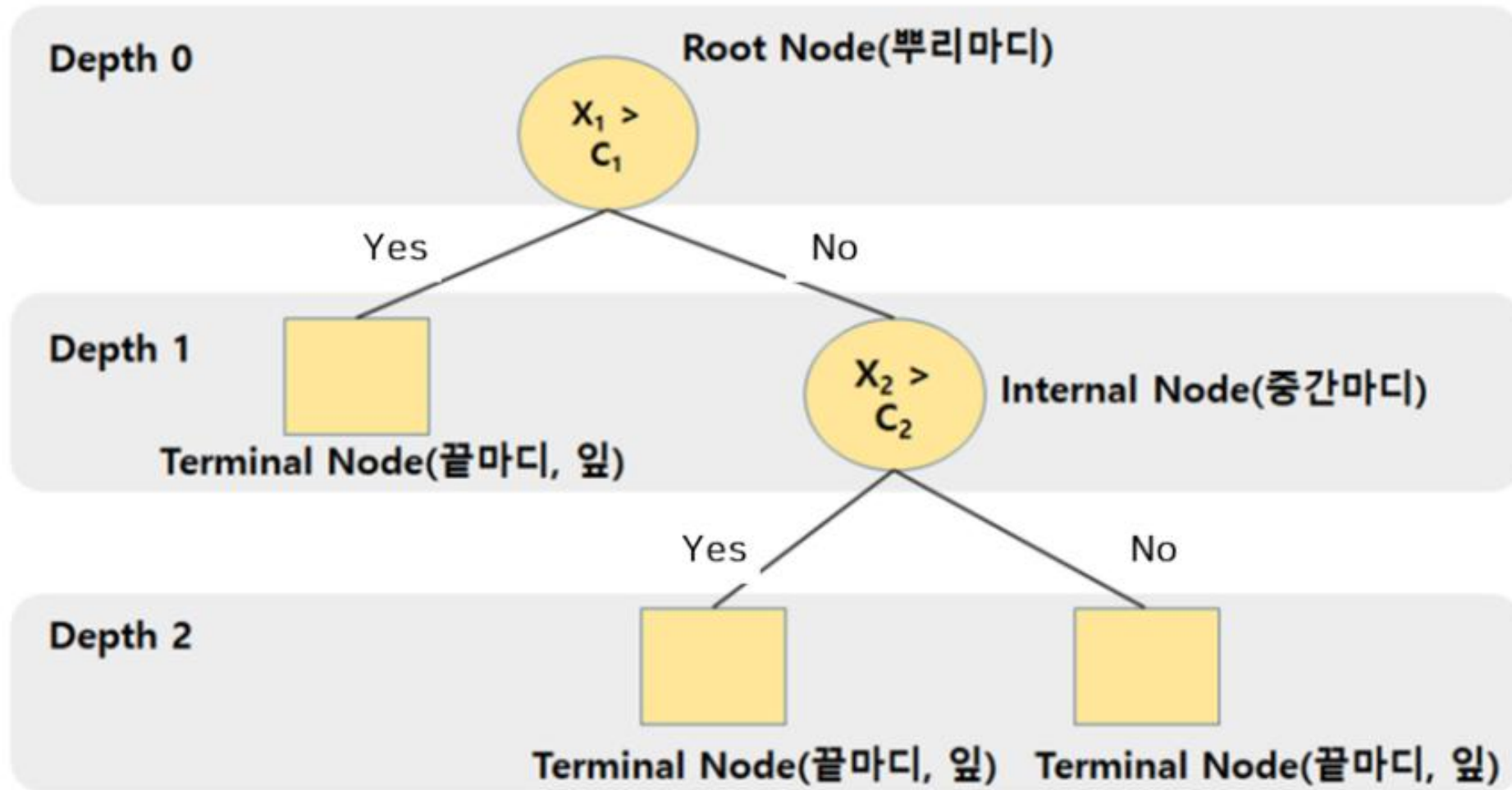
의사결정 나무 알고리즘 구성 요소

의사결정 나무 알고리즘은 의사결정 규칙을 나무(tree) 모양으로 조합하여 목표 변수에 대한 분류 또는 예측을 수행하는 기법이다.

의사결정 나무 알고리즘의 구성 요소는 다음과 같다.

- 뿌리 마디(Root Node): 나무가 시작되는 마디
- 중간 마디(Internal Node): 뿌리마디에서 나온 각 나무줄기 중간에 있는 마디
- 끝 마디(Terminal Node, Leaf Node): 각 나무줄기의 끝에 있는 마디
- 자식 마디(Child Node): 하나의 마디로부터 분리된 2개 이상의 마디
- 부모 마디(Parent Node): 자식마디의 상위 마디
- 가지 (Branch): 하나의 마디로부터 끝 마디까지 연결된 마디들
- 깊이 (Depth): 가지를 이루는 마디의 개수

01 의사결정트리



01 의사결정트리

의사결정 나무 알고리즘 동작 원리

의사결정 나무 알고리즘은 "서로 다른 클래스의 데이터가 잘 구분되게 하는 규칙"으로 분리규칙에 의한 데이터의 불순도 변화량을 사용한다. (다른 분류알고리즘의 경우 다른 기준을 가지고 학습할 수 있다.) 의사결정 나무는 다음과 같은 절차(알고리즘)을 반복적으로 수행하여 학습된 의사결정 기준으로 데이터를 분류한다.

- 1) 변수 선택: 목표변수와 독립변수 선택
- 2) 의사결정나무 형성: 다음과 같은 분리 기준(Split Criterion)과 정지규칙, 평가기준 등으로 훈련데이터를 활용 독립변수의 차원공간을 반복적으로 분할/분리 과정을 통해 의사결정 나무를 형성
- 3) 가지치기: 평가용 데이터를 활용, 부적절한 추론규칙을 가지고 있거나 불필요한 또는 분류오류를 크게 할 위험 마디들을 제거

분리기준 지표	설명
카이제곱 통계량의 p값	분석 대상 자료의 검정통계량의 유의확률 p값이 가장 작은 독립변수와 이에 따른 최적 분리로 Child Node 구성
지니지수 (Gini Index)	데이터의 불순도(Impurity)를 나타내는 지니지수를 측정하여 이를 감소시켜 주는 독립변수와 이에 따른 최적 분리로 Child Node 구성
엔트로피 지수 (Entropy Index)	불순도를 나타내는 지표로서 이를 감소시켜 주는 독립변수와 이에 따른 최적 분리로 Child Node 구성
ANOVA Table의 F 통계량 p값	ANOVA Table의 F 통계량에 대한 유의확률 p값이 가장 작은 독립변수와 이에 따른 최적 분리로 Child 노드를 구성
분산의 감소량	종속변수의 실측치와 모형에 의한 추정치와의 오차를 최소화하는 기준과 같이, 분산을 최소화하는 독립변수와 이에 따른 최적 분리로 Child Node 구성

01 의사결정트리

- 붓꽃 데이터셋을 의사결정트리(DecisionTreeClassifier)를 훈련

```
▶ from sklearn.datasets import load_iris
   from sklearn.tree import DecisionTreeClassifier

   iris = load_iris(as_frame=True)

   X_iris = iris.data[["petal length (cm)", "petal width (cm)"]].values
   y_iris = iris.target

   tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
   tree_clf.fit(X_iris, y_iris)
```

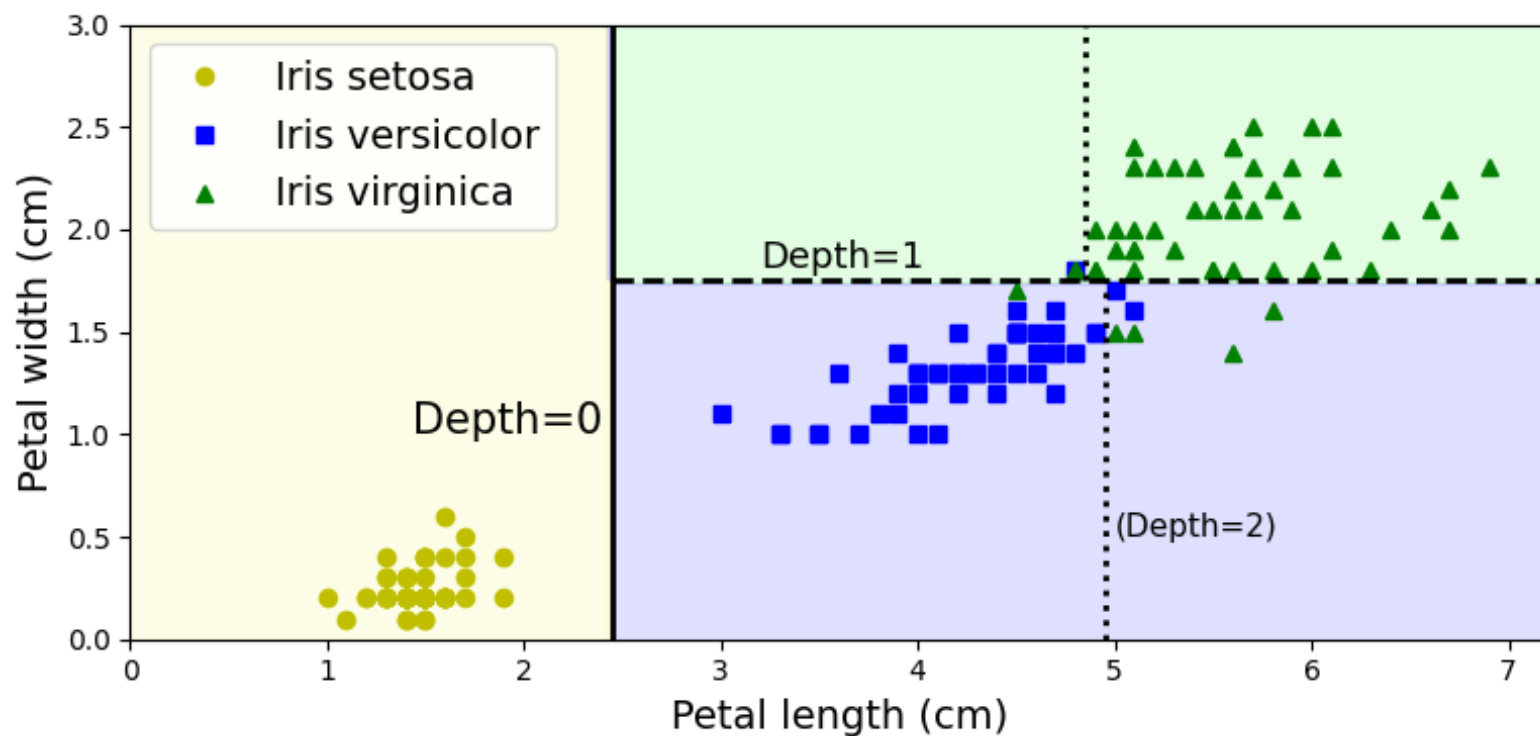
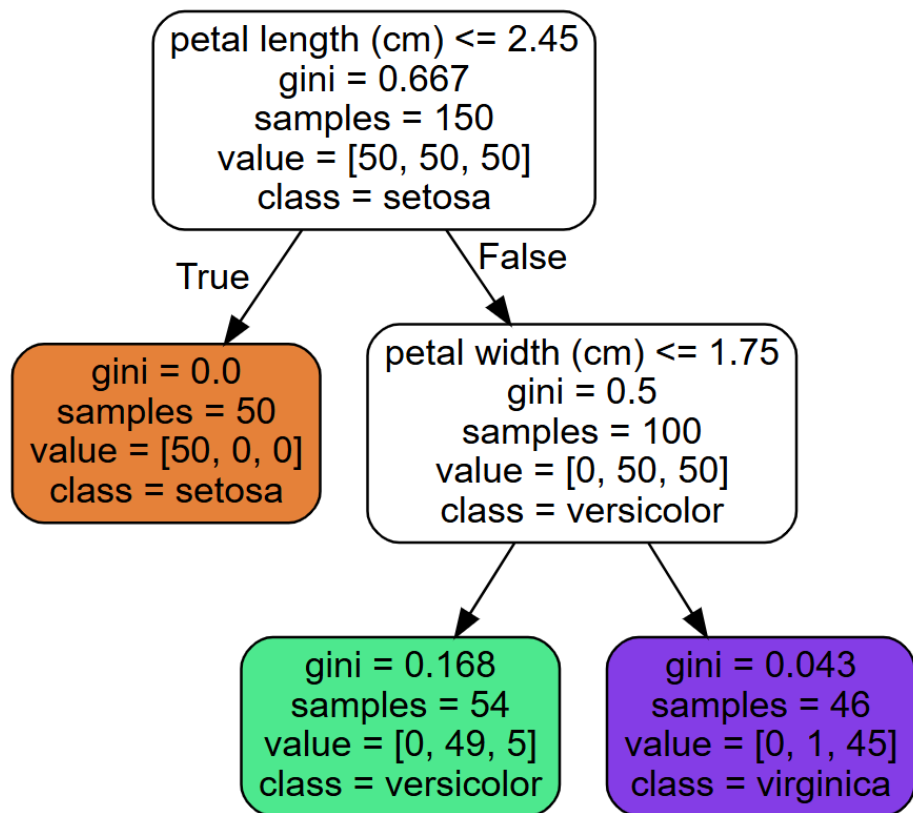
- 의사결정트리 모델 시각화 (export_graphviz())

```
▶ from sklearn.tree import export_graphviz

   export_graphviz( tree_clf, out_file=str("iris_tree.dot"),
                    feature_names=["petal length (cm)", "petal width (cm)"],
                    class_names=iris.target_names, rounded=True, filled=True )
```

01 의사결정트리

첫 번째 결정 트리



01 의사결정트리

■ 노드의 속성

petal length (cm) <= 2.45
gini = 0.667
samples = 150
value = [50, 50, 50]
class = setosa

- sample 속성 - 얼마나 많은 훈련 샘플이 적용되었는지 계산
- value 속성 - 노드에서 각 클래스에 얼마나 많은 훈련 샘플이 있는지 계산
- gini 속성 - 지니 불순도(gini impurity)를 측정

식 6-1 지니 불순도

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

- G_i 는 i 번째 노드의 지니 불순도
- $p_{i,k}$ 는 i 번째 노드에 있는 훈련 샘플 중 클래스 k 에 속한 샘플의 비율

01 의사결정트리

- 결정 트리 훈련
 - CART(classification and regression tree) 알고리즘

식 6-2 분류에 대한 CART 비용 함수

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

여기서 $\begin{cases} G_{\text{left/right}} \text{는 왼쪽 / 오른쪽 서브셋의 불순도} \\ m_{\text{left/right}} \text{는 왼쪽 / 오른쪽 서브셋의 샘플 수} \end{cases}$

01 의사결정트리

- DecisionTreeClassifier 클래스는 지니 불순도를 주로 사용
 - criterion 매개변수를 "entropy"로 지정하여 엔트로피 불순도 사용
 - 지니 불순도가 계산이 빠름
 - 지니 불순도는 가장 빈도 높은 클래스를 한쪽 가지(branch)로 고립시키는 경향이 있음
 - 엔트로피는 균형 잡힌 트리를 생성

식 6-3 엔트로피

$$H_i = - \sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^n p_{i,k} \log_2(p_{i,k})$$

01 의사결정트리

- 분기 기준 지표

분리기준 지표	설명
카이제곱 통계량의 p값	분석 대상 자료의 검정통계량의 유의확률 p값이 가장 작은 독립변수와 이에 따른 최적 분리로 Child Node 구성
지니지수 (Gini Index)	데이터의 불순도(Impurity)를 나타내는 지니지수를 측정하여 이를 감소시켜 주는 독립변수와 이에 따른 최적 분리로 Child Node 구성
엔트로피 지수 (Entropy Index)	불순도를 나타내는 지표로서 이를 감소시켜 주는 독립변수와 이에 따른 최적 분리로 Child Node 구성
ANOVA Table의 F 통계량 p값	ANOVA Table의 F 통계량에 대한 유의확률 p값이 가장 작은 독립변수와 이에 따른 최적 분리로 Child 노드를 구성
분산의 감소량	종속변수의 실측치와 모형에 의한 추정치와의 오차를 최소화하는 기준과 같이, 분산을 최소화하는 독립변수와 이에 따른 최적 분리로 Child Node 구성

01 의사결정트리

- 클래스 확률 추정
 - 결정 트리에서 한 샘플이 특정 클래스 k에 속할 확률을 추정

```
▶ tree_clf.predict_proba([[5, 1.5]]).round(3)
```

```
↪ array([[0.    , 0.907, 0.093]])
```

- setosa 0, versicolor 0.907, virginica 0.093

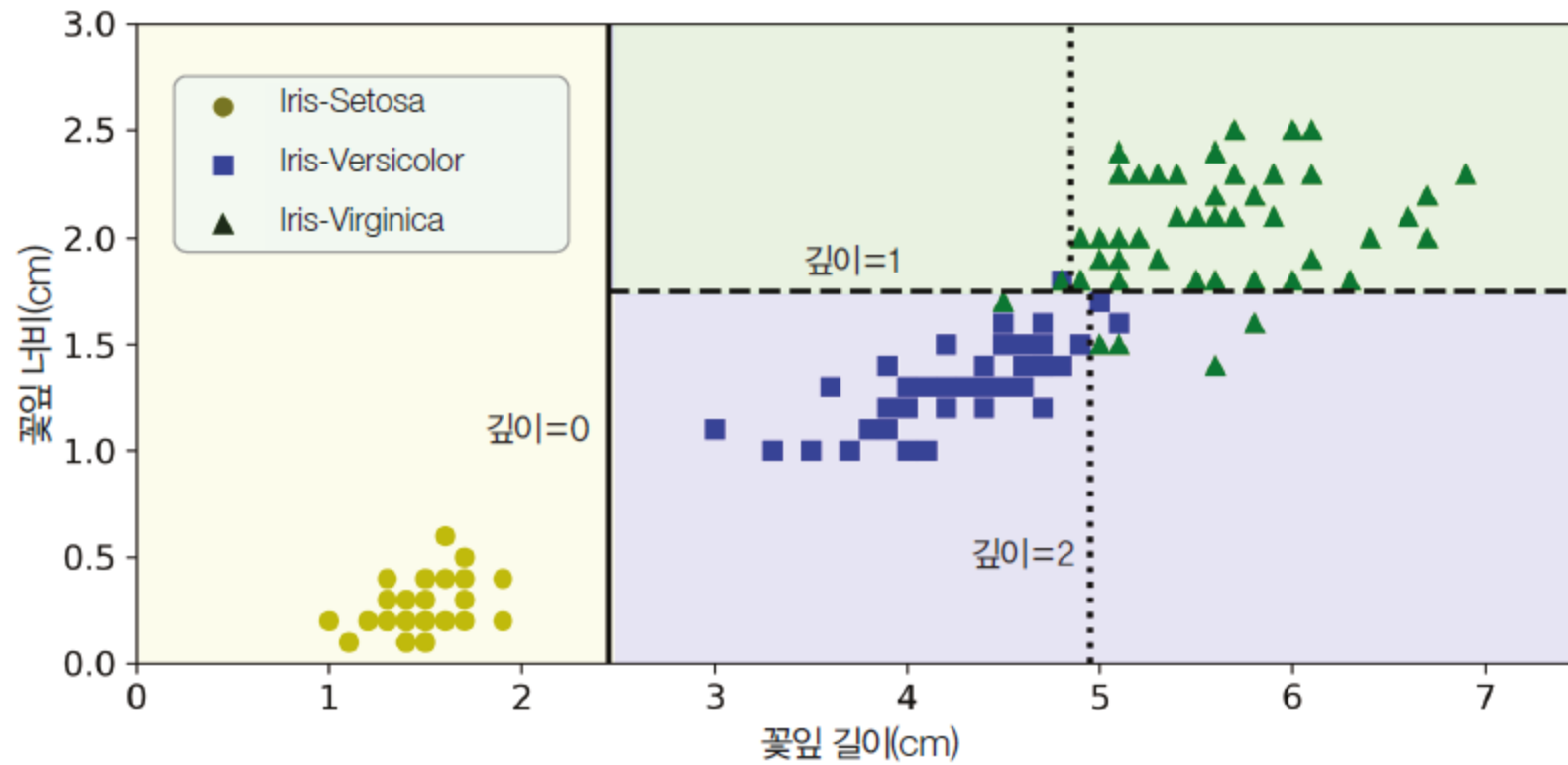
```
▶ tree_clf.predict([[5, 1.5]])
```

```
↪ array([1])
```

- versicolor 로 예측

01 의사결정트리

- 결정 트리의 결정 경계



의사결정트리의 결정 경계

01 의사결정트리

■ 모델 해석: 화이트박스과 블랙박스

■ 화이트박스(white box) 모델

- 의사결정 트리와 같이 직관적이고 결정 방식을 이해하기 쉬운 모델

■ 블랙박스(black box) 모델

- 랜덤 포레스트나 신경망 등의 알고리즘 모델
- 성능이 뛰어나고 예측을 만드는 연산 과정을 쉽게 확인할 수 있음
- 그렇지만 왜 그런 예측을 만드는지는 쉽게 설명하기 어려움

■ 의사결정 트리는 비교적 간단하고 명확한 분류 방법을 사용

■ 해석 가능한 ML(interpretable ML) 분야는 사람이 이해할 수 있는 방식으로 모델의 결정을 설명할 수 있는 ML 시스템을 만드는 것을 목표

- 시스템이 불공정한 결정을 내리지 않도록 하는 등 많은 영역에서 중요

01 의사결정트리

- 파라미터 모델(parametric model)
 - 매개변수 수가 일정함
 - 선형회귀, 로지스틱 회귀
- 비파라미터 모델(nonparametric model)
 - 고정된 수의 파라미터가 없음
 - 의사결정 트리, K-NN, 랜덤포레스트, SVM, 신경망
- DecisionTreeClassifier에서 결정 트리 형태를 제한하는 하이퍼파라미터

파라미터	설명
max_features	각 노드에서 분할에 사용할 특성(feature)의 최대 수
max_leaf_nodes	리프 노드의 최대 수
min_samples_split	분할되기 위해 노드가 가져야 하는 최소 샘플 수
min_samples_leaf	리프 노드가 생성되기 위해 가지고 있어야 할 최소 샘플 수
min_weight_fraction_leaf	min_samples_leaf와 같지만 가중치가 부여된 전체 샘플 수에서의 비율

01 의사결정트리

■ 파라미터 매개변수

```
▶ from sklearn.datasets import make_moons
```

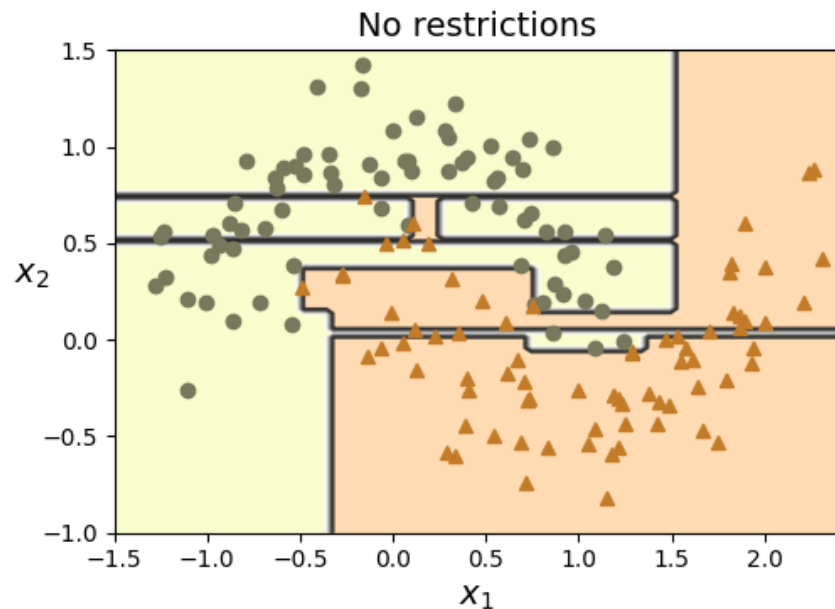
```
X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)
```

```
tree_clf1 = DecisionTreeClassifier(random_state=42)
```

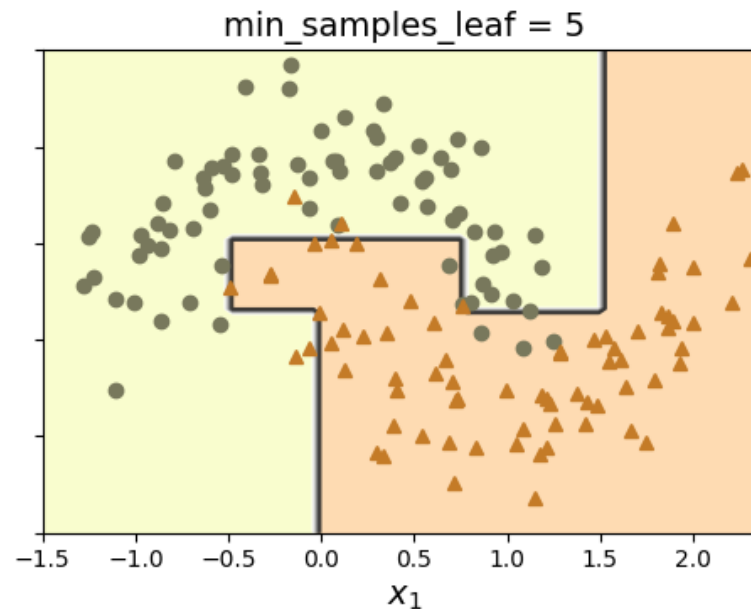
```
tree_clf2 = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
```

```
tree_clf1.fit(X_moons, y_moons)
```

```
tree_clf2.fit(X_moons, y_moons)
```



tree_clf1



tree_clf2

01 의사결정트리

- moons 데이터셋에서 파라미터 테스트

```
▶ from sklearn.datasets import make_moons

X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)

tree_clf1 = DecisionTreeClassifier(random_state=42)
tree_clf2 = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
tree_clf1.fit(X_moons, y_moons)
tree_clf2.fit(X_moons, y_moons)
```

01 의사결정트리

```

▶ def plot_decision_boundary(clf, X, y, axes, cmap):
    x1, x2 = np.meshgrid(np.linspace(axes[0], axes[1], 100),
                          np.linspace(axes[2], axes[3], 100))
    X_new = np.c_[x1.ravel(), x2.ravel()]
    y_pred = clf.predict(X_new).reshape(x1.shape)

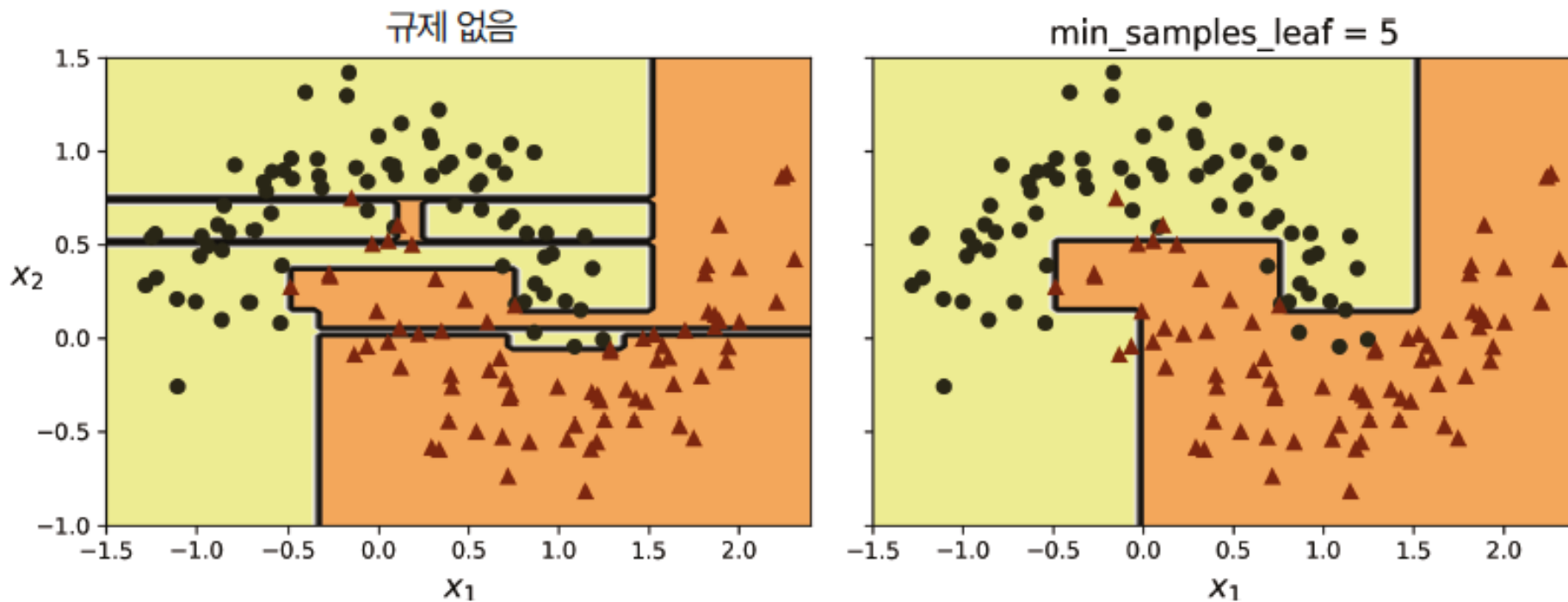
    plt.contourf(x1, x2, y_pred, alpha=0.3, cmap=cmap)
    plt.contour(x1, x2, y_pred, cmap="Greys", alpha=0.8)
    colors = {"Wistia": ["#78785c", "#c47b27"], "Pastel1": ["red", "blue"]}
    markers = ("o", "^")
    for idx in (0, 1):
        plt.plot(X[:, 0][y == idx], X[:, 1][y == idx],
                 color=colors[cmap][idx], marker=markers[idx], linestyle="none")
    plt.axis(axes)
    plt.xlabel(r"$x_1$")
    plt.ylabel(r"$x_2$", rotation=0)

fig, axes = plt.subplots(ncols=2, figsize=(10, 4), sharey=True)
plt.sca(axes[0])
plot_decision_boundary(tree_clf1, X_moons, y_moons,
                      axes=[-1.5, 2.4, -1, 1.5], cmap="Wistia")
plt.title("No restrictions")
plt.sca(axes[1])
plot_decision_boundary(tree_clf2, X_moons, y_moons,
                      axes=[-1.5, 2.4, -1, 1.5], cmap="Wistia")
plt.title(f"min_samples_leaf = {tree_clf2.min_samples_leaf}")
plt.ylabel("")
plt.show()

```

01 의사결정트리

- moons 데이터셋에서 규제 테스트



파라미터 규제 없는 결정 트리(왼쪽)와 파라미터 규제를 추가한 결정 트리(오른쪽)

01 의사결정트리

- 다른 랜덤 시드로 생성한 테스트 세트에서 두 결정 트리를 평가

```
▶ X_moons_test, y_moons_test = make_moons(n_samples=1000, noise=0.2,
                                           random_state=43)
   tree_clf1.score(X_moons_test, y_moons_test)
```

⇒ 0.898

```
▶ tree_clf2.score(X_moons_test, y_moons_test)
```

⇒ 0.92

02 앙상블과 배깅

02 앙상블과 배깅

- 앙상블 학습
 - 여러 개의 개별 모델(학습기, learner)을 결합하여 하나의 더 강력한 모델을 만드는 방법
 - 랜덤 포레스트는 결정 트리의 앙상블
- 투표 기반 분류기
 - 정확도가 80%인 분류기 여러 개를 훈련시켰다고 가정: 로지스틱 회귀 분류기, SVM 분류기, 랜덤 포레스트 분류기, K-최근접 이웃 분류기 등

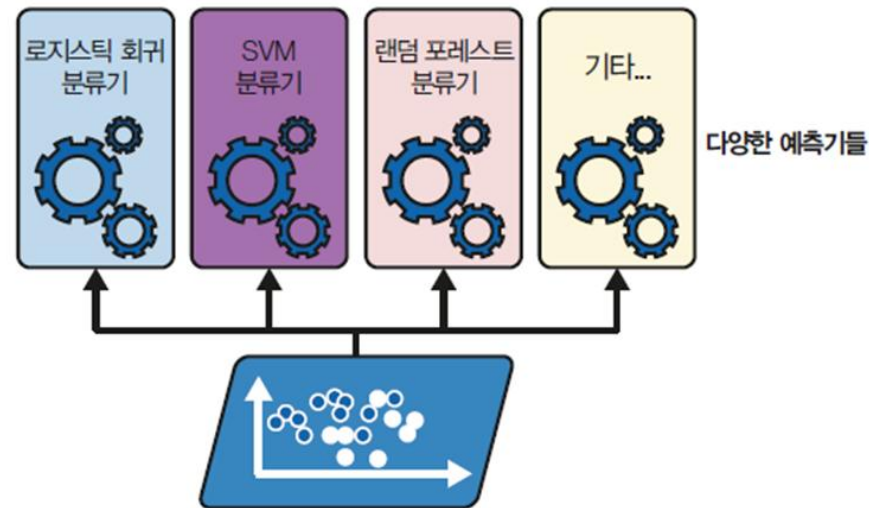


그림 7-1 여러 분류기 훈련시키기

02 앙상블과 배깅

- 직접 투표(hard voting) 분류기
 - 강한 학습기
 - 단독으로도 정확도가 높은 모델
 - 약한 학습기
 - 정확도가 50%보다 약간 높은 모델

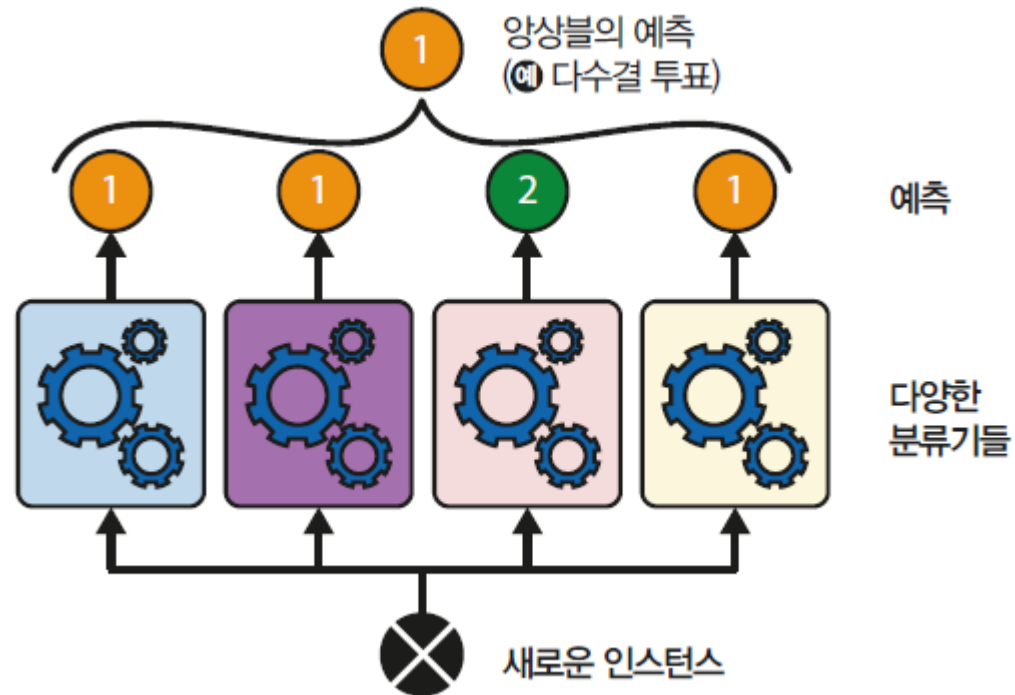


그림 7-2 직접 투표 분류기의 예측

02 앙상블과 배깅

- 큰 수의 법칙(law of large numbers)

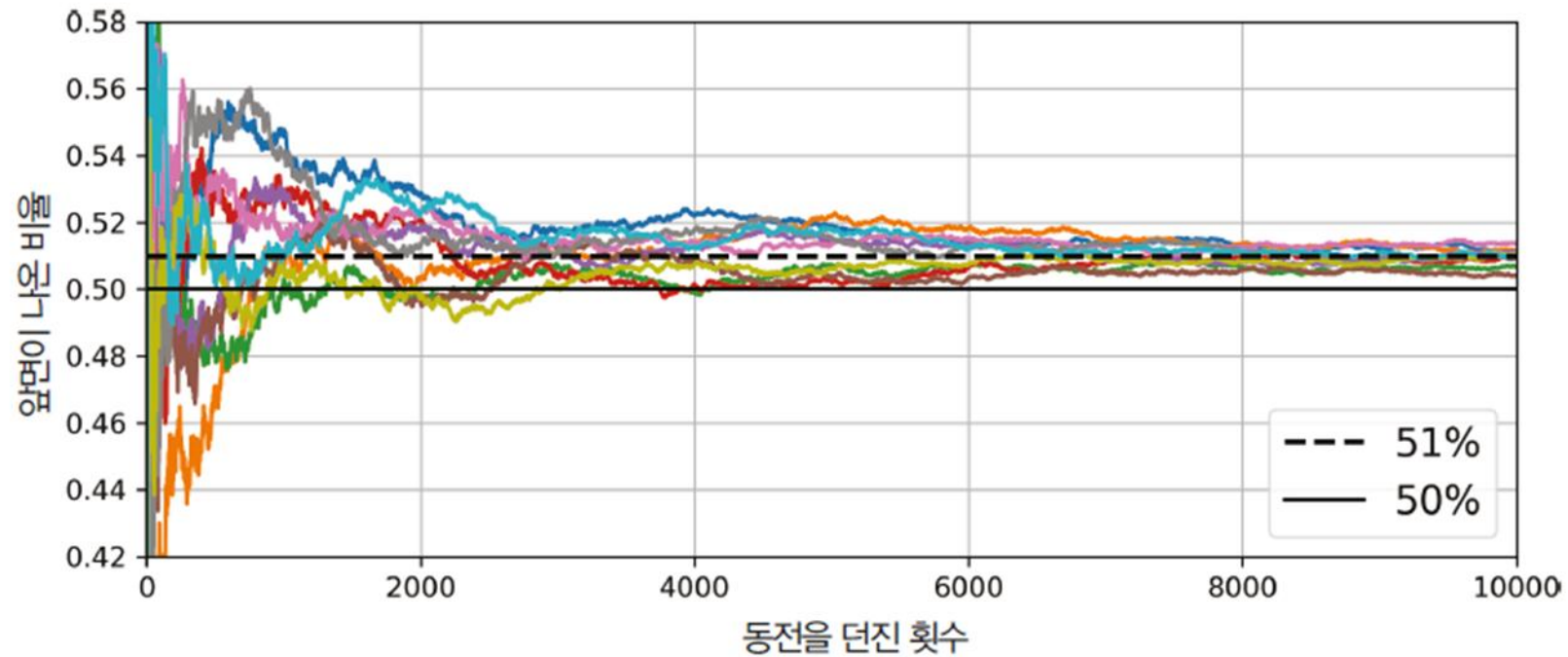


그림 7-3 큰 수의 법칙

02 앙상블과 배깅

- moons 데이터셋을 로드하고 훈련 세트와 테스트 세트로 분할한 다음 세 가지 다양한 분류기로 구성된 투표 기반 분류기를 생성하고 훈련
 - VotingClassifier 클래스 사용

```

▶ from sklearn.datasets import make_moons
   from sklearn.ensemble import RandomForestClassifier, VotingClassifier
   from sklearn.linear_model import LogisticRegression
   from sklearn.model_selection import train_test_split
   from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(random_state=42))
    ]
)
voting_clf.fit(X_train, y_train)

```

02 앙상블과 배깅

- 테스트 세트에서 훈련된 각 분류기의 정확도 확인

```
▶ for name, clf in voting_clf.named_estimators_.items():  
    print(name, "=", clf.score(X_test, y_test))
```

```
⇒ lr = 0.864  
   rf = 0.896  
   svc = 0.896
```

- 투표 기반 분류기의 predict() 메서드를 호출하여 직접 투표를 수행

```
▶ voting_clf.predict(X_test[:1])
```

```
⇒ array([1])
```

```
▶ [clf.predict(X_test[:1]) for clf in voting_clf.estimators_]
```

```
⇒ [array([1]), array([1]), array([0])]
```

- 테스트 세트에서 투표 기반 분류기의 성능

```
▶ voting_clf.score(X_test, y_test)
```

```
⇒ 0.912
```

02 앙상블과 배깅

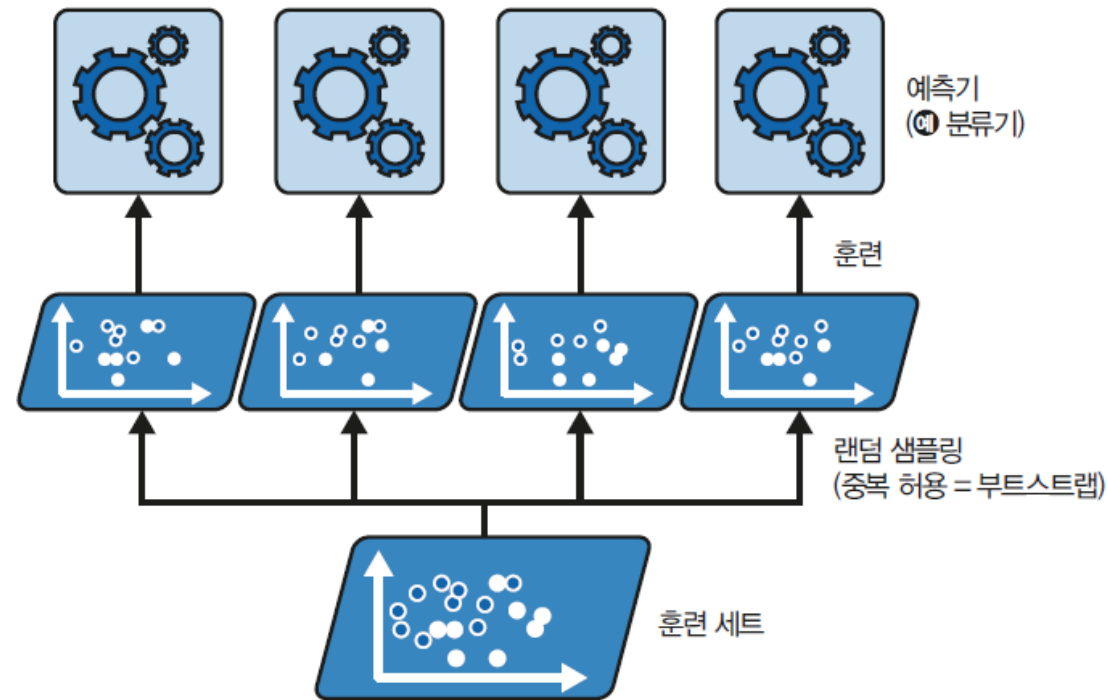
- 간접 투표(soft voting)
 - 각 분류기가 클래스의 확률을 예측하여 (즉, predict_proba() 메서드 이용) 개별 분류기의 예측을 평균 내어 확률이 가장 높은 클래스를 예측
 - 투표 기반 분류기의 voting 매개변수를 "soft"로 바꾸고 모든 분류기가 클래스의 확률을 추정

```
▶ voting_clf.voting = "soft"
  voting_clf.named_estimators["svc"].probability = True
  voting_clf.fit(X_train, y_train)
  voting_clf.score(X_test, y_test)
```

↔ 0.92

02 앙상블과 배깅

- 배깅(bagging, bootstrap aggregating의 줄임말)
 - 훈련 세트에서 중복을 허용하여 샘플링하는 방식
- 페이스팅(pasting)
 - 중복을 허용하지 않고 샘플링하는 방식



배깅과 페이스팅은 훈련 세트에서 랜덤으로 샘플링하여 여러 개의 예측기를 훈련

02 앙상블과 배깅

■ 사이킷런의 배깅과 페이스팅

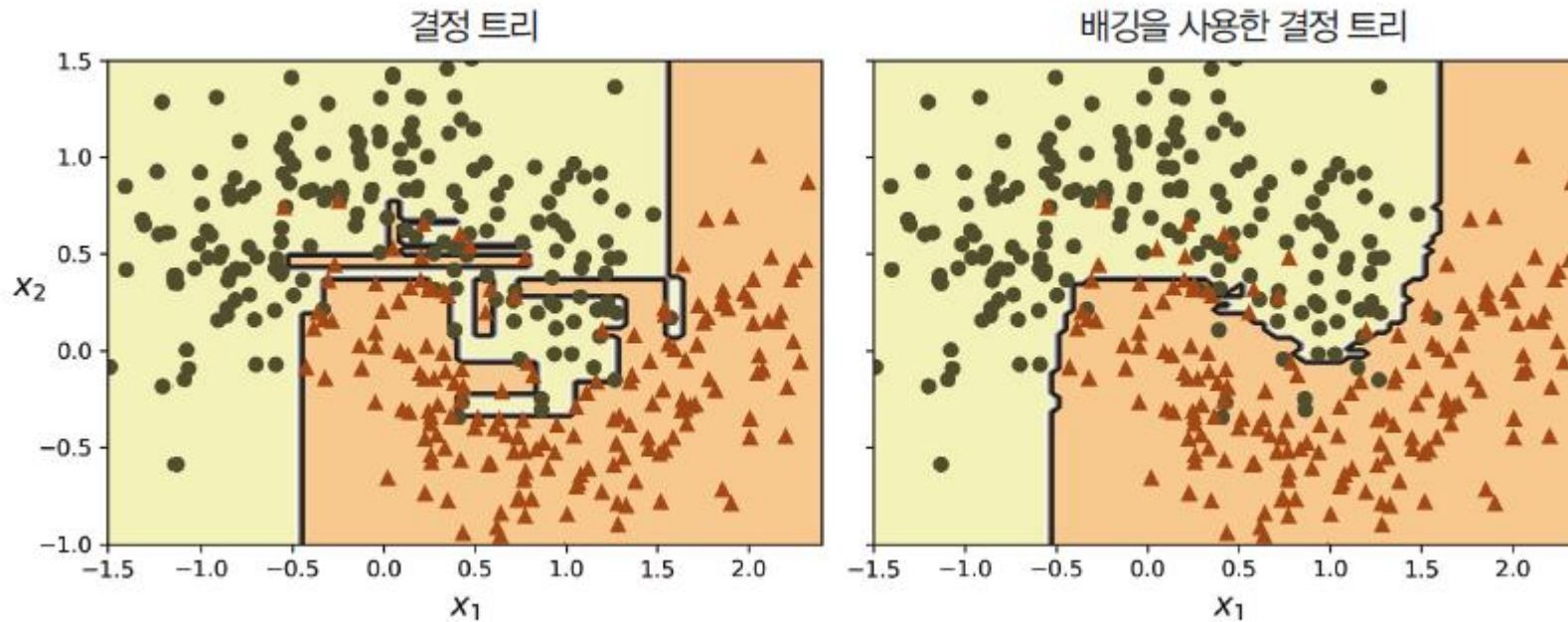
■ BaggingClassifier

```
▶ from sklearn.ensemble import BaggingClassifier
   from sklearn.tree import DecisionTreeClassifier

   bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
                               max_samples=100, n_jobs=-1, random_state=42)
   bag_clf.fit(X_train, y_train)
```


02 앙상블과 배깅

- 단일 결정 트리 결정 경계와 500개의 트리를 사용한 배깅 앙상블의 결정 경계 비교



단일 결정 트리(왼쪽)와 500개의 트리로 만든 배깅 앙상블(오른쪽) 비교

02 앙상블과 배깅

```

▶ def plot_decision_boundary(clf, X, y, alpha=1.0):
    axes=[-1.5, 2.4, -1, 1.5]
    x1, x2 = np.meshgrid(np.linspace(axes[0], axes[1], 100),
                        np.linspace(axes[2], axes[3], 100))
    X_new = np.c_[x1.ravel(), x2.ravel()]
    y_pred = clf.predict(X_new).reshape(x1.shape)

    plt.contourf(x1, x2, y_pred, alpha=0.3 * alpha, cmap='Wistia')
    plt.contour(x1, x2, y_pred, cmap="Greys", alpha=0.8 * alpha)
    colors = ["#78785c", "#c47b27"]
    markers = ("o", "^")
    for idx in (0, 1):
        plt.plot(X[:, 0][y == idx], X[:, 1][y == idx],
                color=colors[idx], marker=markers[idx], linestyle="none")
    plt.axis(axes)
    plt.xlabel(r"$x_1$")
    plt.ylabel(r"$x_2$", rotation=0)

tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)

fig, axes = plt.subplots(ncols=2, figsize=(10, 4), sharey=True)
plt.sca(axes[0])
plot_decision_boundary(tree_clf, X_train, y_train)
plt.title("결정 트리")
plt.sca(axes[1])
plot_decision_boundary(bag_clf, X_train, y_train)
plt.title("배깅을 사용한 결정 트리")
plt.ylabel("")
plt.show()

```

02 앙상블과 배깅

■ OOB 평가

- BaggingClassifier는 중복 허용이 디폴트(bootstrap=True) 훈련 세트의 크기인 m개 샘플을 선택
 - 복원추출로 인해 각 예측기에 훈련 샘플의 63% 정도가 평균적으로 샘플링
 - 각 샘플이 한번이라도 선택될 확률 $1 - \left(1 - \frac{1}{m}\right)^m = 1 - e^{-1} \approx 0.632$
 - OOB(out-of-bag) 샘플 - 선택되지 않은 나머지 37% 샘플
- oob_score=True로 지정하면 훈련이 끝난 후 자동으로 OOB 평가를 수행

```
▶ bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
                             oob_score=True, n_jobs=-1, random_state=42)
   bag_clf.fit(X_train, y_train)
   bag_clf.oob_score_
```

⇒ 0.896

02 앙상블과 배깅

- OOB 평가 결과 확인

```
▶ from sklearn.metrics import accuracy_score
```

```
y_pred = bag_clf.predict(X_test)
accuracy_score(y_test, y_pred)
```

```
↪ 0.92
```

- OOB 샘플에 대한 결정 함수의 값도 oob_decision_function_ 변수에서 확인

```
▶ bag_clf.oob_decision_function_[:3]
```

```
↪ array([[0.32352941, 0.67647059],
        [0.3375      , 0.6625      ],
        [1.          , 0.          ]])
```

03 랜덤포레스트와 XGBoost

03 랜덤포레스트와 XGBoost

- **랜덤 패치 방식**(random patches method)
 - 훈련 샘플과 특성을 모두 랜덤하게 샘플링 하는 방법
 - 개별 학습기는 훈련샘플 일부와 특성 일부를 가지고 학습
- **랜덤 서브스페이스 방식**(random subspaces method)
 - 훈련 샘플은 모두 사용
 - **특성을 샘플링**
- 특성 샘플링은 서로다른 feature 조합으로 다양한 모델 생성 가능

03 랜덤포레스트와 XGBoost

- 500개의 트리로 이뤄진 랜덤 포레스트 분류기를 가능한 모든 CPU 코어에서 훈련시키는 코드

```
▶ from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
                                n_jobs=-1, random_state=42)

rnd_clf.fit(X_train, y_train)
y_pred_rf = rnd_clf.predict(X_test)
```

- 랜덤 포레스트 알고리즘은 트리의 노드를 분할 시 **랜덤으로 선택한 특성 후보 중에서 최적의 특성을 찾는 방식**으로 무작위성을 더 주입
 - 기본적으로 $\sqrt{n}$ 개의 특성을 선택

```
▶ bag_clf = BaggingClassifier(
    DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),
    n_estimators=500, n_jobs=-1, random_state=42)
```

03 랜덤포레스트와 XGBoost

■ 특성 중요도

- 랜덤 포레스트의 장점은 특성의 상대적 중요도 측정이 쉽다
- 훈련이 끝난 뒤 특성마다 중요도를 계산하고 정규화
 - 이 값은 feature_importances_ 변수에 저장
- iris 데이터셋에 RandomForestClassifier를 훈련하여 각 특성의 중요도 확인 결과

```
▶ from sklearn.datasets import load_iris

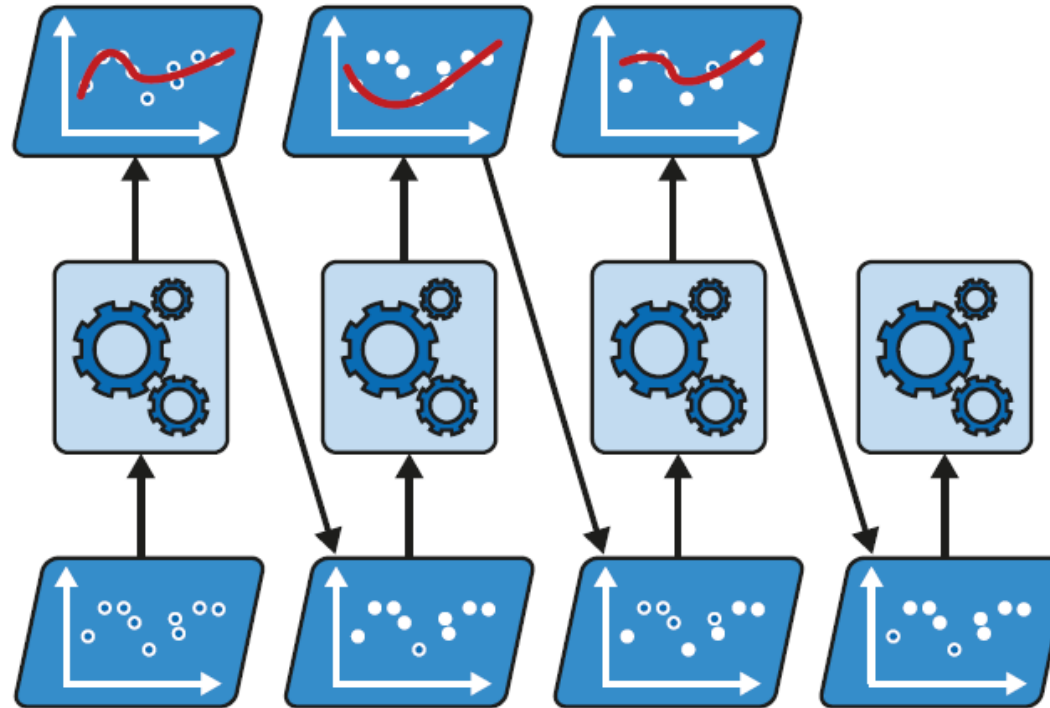
iris = load_iris(as_frame=True)
rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
rnd_clf.fit(iris.data, iris.target)
for score, name in zip(rnd_clf.feature_importances_, iris.data.columns):
    print(round(score, 2), name)
```

```
⇒ 0.11 sepal length (cm)
   0.02 sepal width (cm)
   0.44 petal length (cm)
   0.42 petal width (cm)
```


03 랜덤포레스트와 XGBoost

■ AdaBoost

- 이전 모델이 과소적합했던 훈련 샘플의 가중치를 더 높임



샘플의 가중치를 업데이트하면서 순차적으로 학습하는 AdaBoost

03 랜덤포레스트와 XGBoost

- moons 데이터셋에 훈련시킨 다섯 개의 연속된 예측기의 결정 경계
 - 이 모델은 규제를 강하게 한 RBF 커널 SVM 분류기

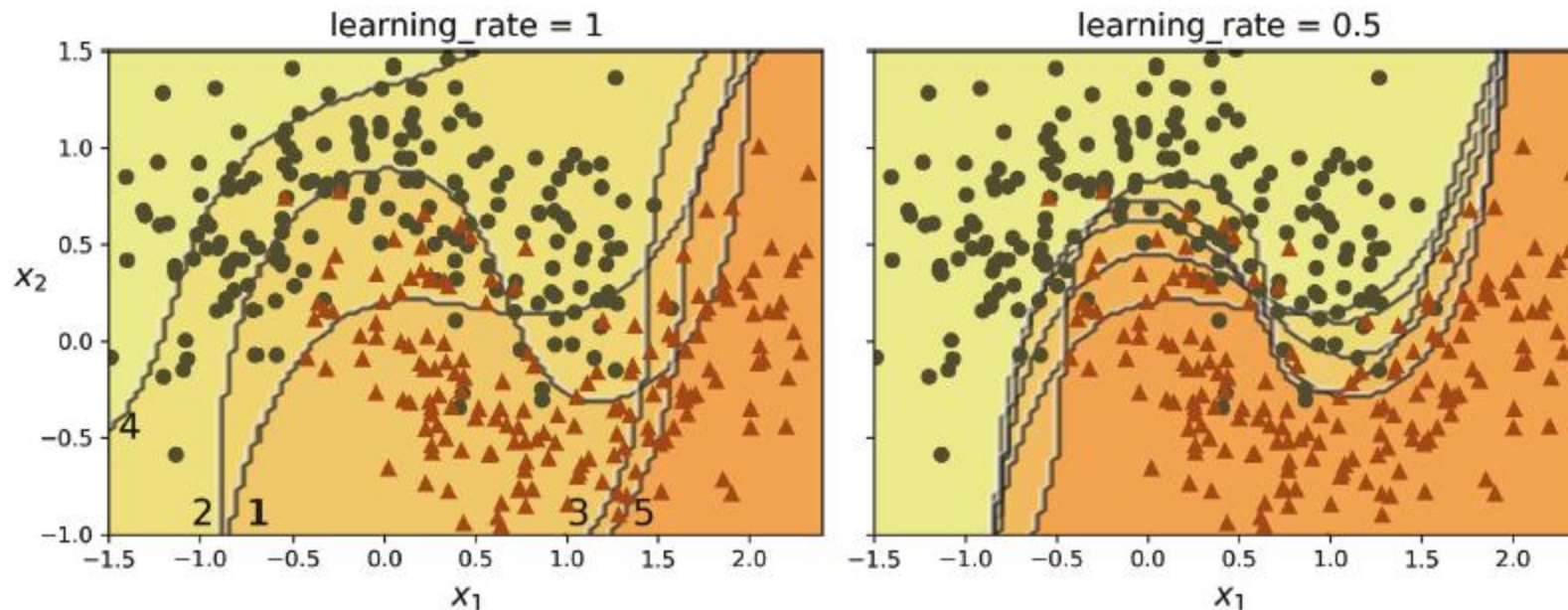


그림 7-8 연속된 예측기의 결정 경계

03 랜덤포레스트와 XGBoost

- 가중치가 적용된 훈련 세트의 전반적인 정확도에 따라 예측기마다 다른 가중치가 적용

식 7-1 j번째 예측기의 가중치가 적용된 오류율

$$r_j = \frac{\sum_{i=1}^m \mathbb{1}_{\hat{y}_j^{(i)} \neq y^{(i)}}}{\sum_{i=1}^m 1} \quad \text{여기서 } \hat{y}_j^{(i)} \text{는 } i\text{번째 샘플에 대한 } j\text{번째 예측기의 예측}$$

- 예측기가 정확할수록 가중치는 더 높아짐

식 7-2 예측기 가중치

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

03 랜덤포레스트와 XGBoost

- AdaBoost 알고리즘에서는 아래 규칙을 사용해 샘플의 가중치를 업데이트
 - 즉, 잘못 분류된 샘플의 가중치가 높아짐

가중치 업데이트 규칙

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \hat{y}_j^{(i)} = y^{(i)} \text{일 때} \\ w^{(i)} \exp(\alpha_j) & \hat{y}_j^{(i)} \neq y^{(i)} \text{일 때} \end{cases} \quad \text{여기서 } i = 1, 2, \dots, m$$

- 모든 샘플의 가중치를 정규화합니다(즉, $\sum_{i=1}^m w^{(i)}$ 로 나눔)
- 새 예측기가 업데이트된 가중치를 사용해 훈련
- 가중치 합이 가장 큰 클래스가 예측 결과

식 7-4 AdaBoost 예측

$$\hat{y}(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \sum_{\substack{j=1 \\ \hat{y}_j(\mathbf{x})=k}}^N \alpha_j \quad \text{여기서 } N \text{은 예측기 수}$$

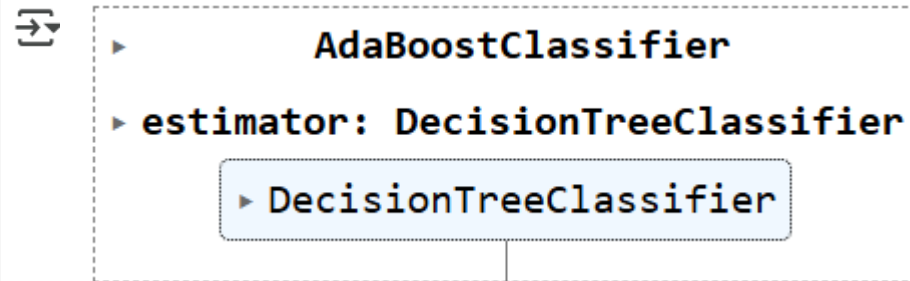
03 랜덤포레스트와 XGBoost

- AdaBoostClassifier를 사용하여 200개의 아주 얇은 결정 트리를 기반으로 하는 AdaBoost 분류기를 훈련
 - 결정 트리는 max_depth=1

```

▶ from sklearn.ensemble import AdaBoostClassifier

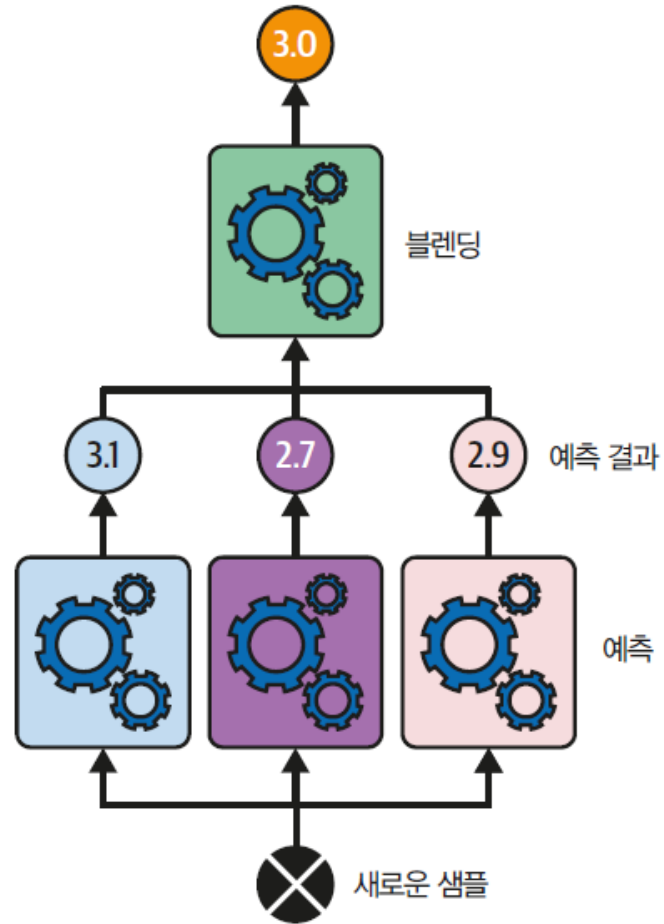
ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=30,
    learning_rate=0.5, random_state=42)
ada_clf.fit(X_train, y_train)
    
```



04 스택킹

04 스택킹

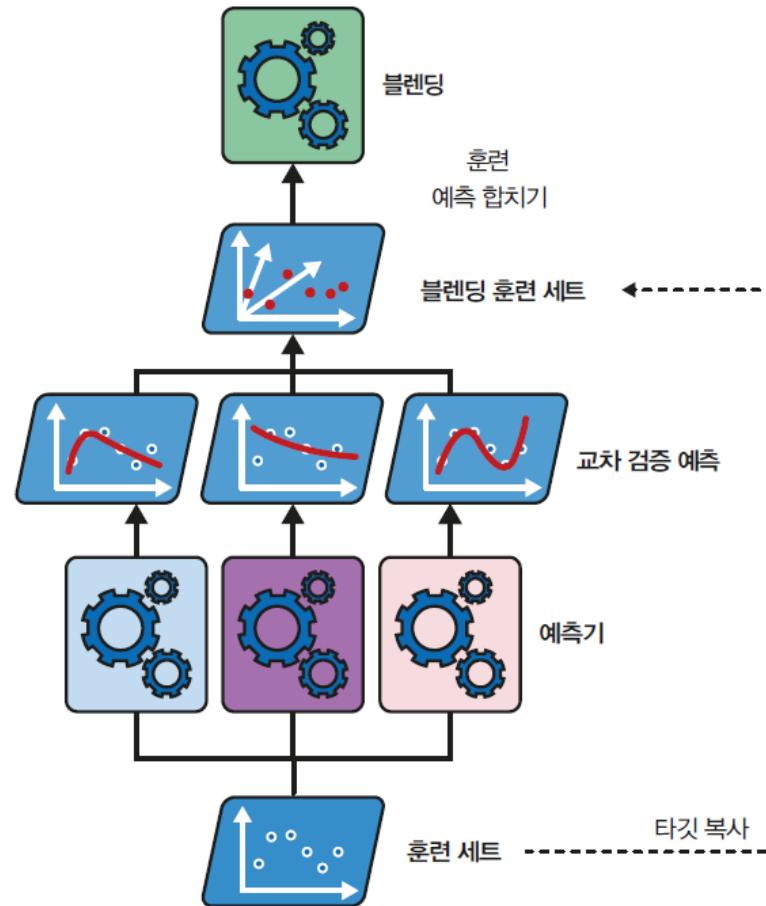
- 블렌더(blender) 또는 메타 학습기(meta learner)



블렌딩 예측기를 사용한 예측 취합

04 스택킹

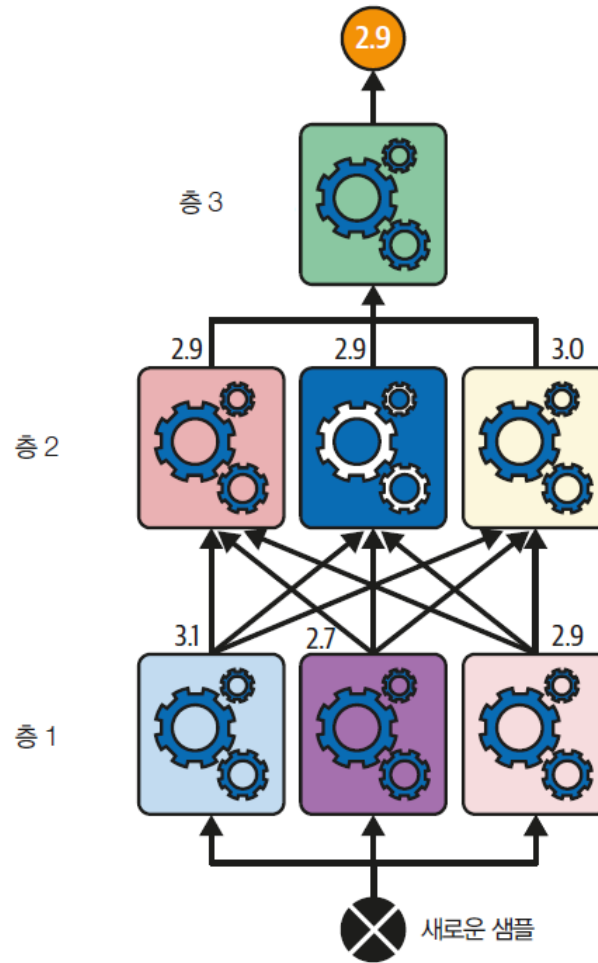
- 블렌더가 학습되면 기본 예측기는 전체 원본 훈련 세트로 마지막에 한 번 더 재훈련



스태킹 앙상블에서 블렌더 훈련하기

04 스택킹

- 여러 블렌더(선형 회귀, 랜덤 포레스트 등)를 훈련하여 전체 블렌더 계층을 얻은 다음 그 위에 다른 블렌더를 추가하여 최종 예측을 생성



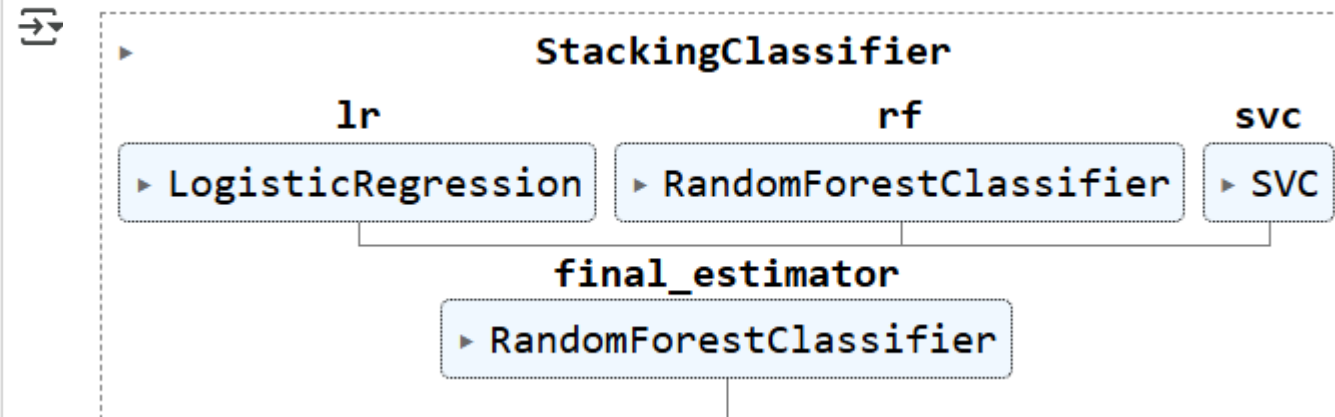
다층 스택킹 앙상블

03 랜덤포레스트와 XGBoost

- 스태킹 앙상블을 위한 StackingClassifier 클래스

```
▶ from sklearn.ensemble import StackingClassifier

stacking_clf = StackingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(probability=True, random_state=42))
    ],
    final_estimator=RandomForestClassifier(random_state=43),
    cv=5 # 교차 검증 폴드 수
)
stacking_clf.fit(X_train, y_train)
```



03 랜덤포레스트와 XGBoost

- 스택킹 앙상블을 위한 StackingClassifier 클래스 결과

```
stacking_clf.score(X_test, y_test)
```

```
0.928
```

04 실습

04 실습

- Wine 데이터 분석

```
# 필요한 라이브러리 로드
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import cross_validate
from sklearn.model_selection import train_test_split
# scikit-learn toy dataset 로드
# Classification 문제를 위한 와인 종류 분류 데이터셋을 로드
wine = datasets.load_wine()
# 데이터 프레임으로 변환
df = pd.DataFrame(data=wine.data, columns=wine.feature_names)
df['target'] = wine.target
df
```

04 실습

■ Wine 데이터 분석

```
print(wine.DESCR)
```

```

>>> .. _wine_dataset:

Wine recognition dataset
-----

**Data Set Characteristics:**

: Number of Instances: 178
: Number of Attributes: 13 numeric, predictive attributes and the class
: Attribute Information:
  - Alcohol
  - Malic acid
  - Ash
  - Alcalinity of ash
  - Magnesium
  - Total phenols
  - Flavanoids
  - Nonflavanoid phenols
  - Proanthocyanins
  - Color intensity
  - Hue
  - OD280/OD315 of diluted wines
  - Proline

- class:
  - class_0
  - class_1
  - class_2

: Summary Statistics:

=====
              Min      Max      Mean      SD
-----

```

04 실습

- Wine 데이터 분석

```
# 모델의 이해를 위해 Feature를 2개만 사용
X = df[['alcohol', 'malic_acid']]
y = df['target']
```

```
# 학습/테스트 데이터셋 분리
train_X, test_X, train_y, test_y = train_test_split(X,
y, test_size=0.2, random_state=42)
```

```
# 학습/평가 데이터 분리
train_sub_X, val_X, train_sub_y, val_y = train_test_split(train_X,
train_y, test_size=0.4, random_state=42)
```

```
# 모델 학습 (max_depth=2일 때)
model = DecisionTreeClassifier(max_depth=2, random_state=42)
model.fit(train_sub_X, train_sub_y)
```

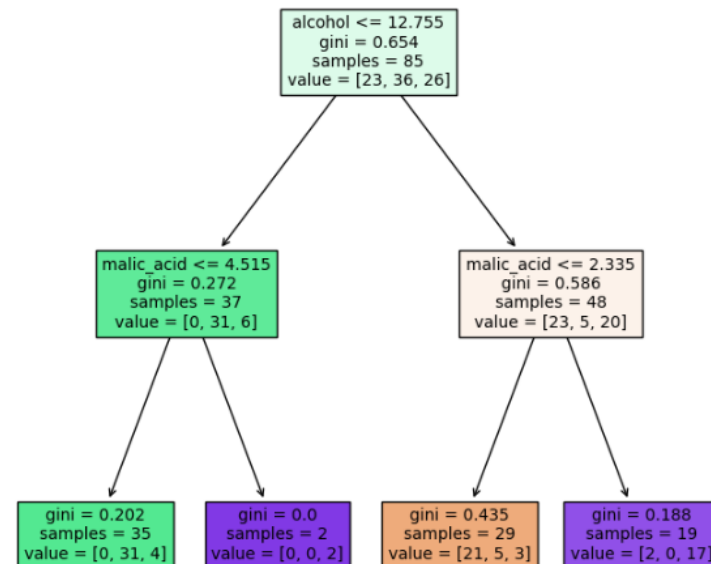
04 실습

Wine 데이터 분석

의사결정 나무 시각화

```
from sklearn.tree import plot_tree
plt.figure(figsize=(8,8))
plot_tree(model, filled=True, feature_names=['alcohol', 'malic_acid'])
```

```
[Text(0.5, 0.8333333333333334, 'alcohol <= 12.755#gini = 0.654#nsamples = 85#nvalue = [23, 36, 26]'),
Text(0.25, 0.5, 'malic_acid <= 4.515#gini = 0.272#nsamples = 37#nvalue = [0, 31, 6]'),
Text(0.125, 0.16666666666666666, 'gini = 0.202#nsamples = 35#nvalue = [0, 31, 4]'),
Text(0.375, 0.16666666666666666, 'gini = 0.0#nsamples = 2#nvalue = [0, 0, 2]'),
Text(0.75, 0.5, 'malic_acid <= 2.335#gini = 0.586#nsamples = 48#nvalue = [23, 5, 20]'),
Text(0.625, 0.16666666666666666, 'gini = 0.435#nsamples = 29#nvalue = [21, 5, 3]'),
Text(0.875, 0.16666666666666666, 'gini = 0.188#nsamples = 19#nvalue = [2, 0, 17]')]
```



04 실습

■ Wine 데이터 분석

모델 평가

```
print("훈련 데이터셋 accuracy: {}".format(model.score(train_sub_X, train_sub_y)))
print("평가용 데이터셋 accuracy: {}".format(model.score(val_X, val_y)))
print("테스트 데이터셋 accuracy: {}".format(model.score(test_X, test_y)))
```

⇒ 훈련 데이터셋 accuracy: 0.8352941176470589
평가용 데이터셋 accuracy: 0.7719298245614035
테스트 데이터셋 accuracy: 0.8611111111111112

Cross Validation 5-Fold 각각의 결과

```
model_cv = DecisionTreeClassifier(max_depth=2, random_state=42)
scores= cross_validate(model_cv, train_X, train_y)
scores
```

⇒ {'fit_time': array([0.00355792, 0.00229526, 0.0024972 , 0.00221038, 0.00222969]),
'score_time': array([0.00177169, 0.0014894 , 0.0017097 , 0.00148368, 0.00148726]),
'test_score': array([0.75862069, 0.68965517, 0.75 , 0.71428571, 0.82142857])}

Cross Validation 평균 값

```
np.mean(scores['test_score'])
```

⇒ 0.7467980295566502

04 실습

■ Wine 데이터 분석

```
# 모델 학습 (max_depth= None 일 때)
model = DecisionTreeClassifier(random_state=42)
model.fit(train_sub_X, train_sub_y)
print("훈련 데이터셋 accuracy: {}".format(model.score(train_sub_X, train_sub_y)))
print("평가용 데이터셋 accuracy: {}".format(model.score(val_X, val_y)))
print("테스트 데이터셋 accuracy: {}".format(model.score(test_X, test_y)))
```

⇒ 훈련 데이터셋 accuracy: 1.0
평가용 데이터셋 accuracy: 0.6140350877192983
테스트 데이터셋 accuracy: 0.8055555555555556

```
# Grid Search로 최적의 파라미터 찾기
params = {'max_depth':[1, 2, 3, 4, 5]}
gs = GridSearchCV(DecisionTreeClassifier(random_state=42), params)
gs.fit(train_X, train_y)
```

```
# 제시한 하이퍼파라미터 중 최적의 파라미터 확인
gs.best_estimator_
```

```
# Grid Search로 찾은 최적의 파라미터로 모델 선정하기
model = gs.best_estimator_
```

04 실습

- Wine 데이터 분석

모델 평가

```
scores= cross_validate(model, train_X, train_y)
np.mean(scores['test_score'])
```



0.7746305418719212

```
print("테스트 데이터셋 accuracy: {}".format(model.score(test_X, test_y)))
```



테스트 데이터셋 accuracy: 0.8888888888888888

Q & A